fourth edition

PHYSICALLY BASED RENDERING

FROM THEORY TO IMPLEMENTATION

MATT PHARR, WENZEL JAKOB,
AND GREG HUMPHREYS

ABOUT THE AUTHORS

Matt Pharr is a Distinguished Research Scientist at NVIDIA. He previously worked on VR at Google, co-founded Neoptica, which worked on programming models for graphics on heterogeneous CPU+GPU systems; Neoptica was acquired by Intel. Before Neoptica, Matt was in the Software Architecture group at NVIDIA, co-founded Exluna, and worked in Pixar's Rendering R+D group. He received his Ph.D. from the Stanford Graphics Lab, working under the supervision of Pat Hanrahan. He was also the editor of *GPU Gems 2*.

Matt Pharr 是 NVIDIA 的杰出研究科学家。他之前在谷歌从事 VR 工作，联合创立了 Neoptica，致力于异构 CPU+GPU 系统上图形的编程模型； Neooptica 被英特尔收购。在加入 Neoptica 之前，Matt 曾在 NVIDIA 的软件架构小组工作，共同创立了 Exluna，并在 Pixar 的渲染研发小组工作。他获得了博士学位。来自斯坦福图形实验室，在 Pat Hanrahan 的监督下工作。他还是《GPU Gems 2》的编辑。

Wenzel Jakob is an assistant professor at EPFL's School of Computer and Communication Sciences. His research interests revolve around material appearance modeling, rendering algorithms, and the high-dimensional geometry of light paths. Wenzel obtained his Ph.D. at Cornell University under the supervision of Steve Marschner, after which he joined ETH Zürich for postdoctoral studies under the supervision of Olga Sorkine Hornung. Wenzel is also the lead developer of the Mitsuba renderer, a research-oriented rendering system.

Wenzel Jakob 是 EPFL 计算机与通信科学学院的助理教授。他的研究兴趣围绕材料外观建模、渲染算法和光路的高维几何。温泽尔获得博士学位。在 Steve Marschner 的指导下在康奈尔大学获得博士学位，之后在 Olga Sorkine Hornung 的指导下加入苏黎世联邦理工学院进行博士后研究。 Wenzel 也是 Mitsuba 渲染器（一种面向研究的渲染系统）的首席开发人员。

Greg Humphreys is Director of Engineering at FanDuel, having previously worked on the Chrome graphics team at Google and the OptiX GPU raytracing engine at NVIDIA. Before that, he was a professor of Computer Science at the University of Virginia, where he conducted research in both high performance and physically based computer graphics, as well as computer architecture and visualization. Greg has a B.S.E. degree from Princeton, and a Ph.D. in Computer Science from Stanford under the supervision of Pat Hanrahan. When he's not tracing rays, Greg can usually be found playing tournament bridge.

Greg Humphreys 是 FanDuel 的工程总监，之前曾在 Google 的 Chrome 图形团队和 NVIDIA 的 OptiX GPU 光线追踪引擎工作。在此之前，他是弗吉尼亚大学计算机科学教授，在那里他进行了高性能和基于物理的计算机图形以及计算机体系结构和可视化方面的研究。Greg 拥有普林斯顿大学的 BSE 学位和博士学位。在 Pat Hanrahan 的指导下获得斯坦福大学计算机科学博士学位。当格雷格不追踪光线时，通常会发现他正在打桥牌锦标赛。

ABOUT ME

译者序

本书（也就是仓库 pbrt-v4-zh）由 gpt 翻译而来。

本书使用 知识共享署名–非商业性使用–相同方式共享 4.0 国际公共许可协议 进行许可。

由 Remo 修改格式并且润色译文。

翻译进度

PBR Contents

1. Perface ... 15
1.1. Audience ... 17
1.2. Overview and Goals ... 18
1.3. Changes Between The First and Second Editions ... 21
1.4. Changes Between The Second and Third Editions ... 23
1.5. Changes Between The Third and Fourth Editions ... 25
1.6. Acknowledgments ... 27
1.7. Production ... 32
1.8. The Online edition ... 34
1.9. Scenes, Models, and Data ... 35
1.10. About The Cover ... 37
1.11. Further Reading ... 38

1. Introduction ... 39
1.1. Literate Programming ... 41
1.2. Photorealistic Rendering and the Ray-Tracing Algorithm ... 45
1.2.1. Cameras and Film ... 48
1.2.2. Ray-Object Intersections ... 50
1.2.3. Light Distribution ... 53
1.2.4. Visibility ... 55
1.2.5. Light Scattering at Surfaces ... 56
1.2.6. Indirect Light Transport ... 58
1.2.7. Ray Propagation ... 60
1.3. pbrt: System Overview ... 63
1.3.1. Phases of Execution ... 70
1.3.2. pbrt's main() Function ... 70
1.3.3. Integrator Interface ... 73
1.3.4. ImageTileIntegrator and the Main Rendering Loop ... 77
1.3.5. 1.3.5 RayIntegrator Implementation ... 84
1.3.6. 1.3.6 Random Walk Integrator ... 89
1.4. How to Proceed through This Book ... 99
1.4.1. The Exercises ... 101
1.4.2. Viewing the images ... 101
1.4.3. The Online Edition ... 101
1.5. Using and Understanding the Code ... 103
1.5.1. Source Code Organization ... 103
1.5.2. Naming Conventions ... 104
1.5.3. Pointer or Reference? ... 105
1.5.4. Abstraction versus Efficiency ... 105
1.5.5. pstd ... 106
1.5.6. Allocators ... 106
1.5.7. Dynamic Dispatch ... 108
1.5.8. Code Optimization ... 109
1.5.9. Debugging and Logging ... 109
1.5.10. Parallelism and Thread Safety ... 111
1.5.11. Extending the System ... 112
1.5.12. Bugs ... 113
1.6. A Brief History of Physically Based Rendering ... 114
1.6.1. Research ... 114

1.6.2. Production ... 118
1.7. Further Reading ... 122
1.8. Exercises ... 125

2. Monte Carlo Integration ... 126
2.1. Monte Carlo: Basics ... 128
2.1.1. Background and Probability Review ... 128
2.1.2. Expected Values ... 131
2.1.3. The Monte Carlo Estimator ... 132
2.1.4. Error in Monte Carlo Estimators ... 134
2.2. Improving Efficiency ... 138
2.2.1. Stratified Sampling ... 138
2.2.2. Importance Sampling ... 141
2.2.3. Multiple Importance Sampling ... 144
2.2.3.1. MIS Compensation ... 148
2.2.4. Russian Roulette ... 148
2.2.5. Splitting ... 150
2.3. Sampling Using the Inversion Method ... 152
2.3.1. Discrete Case ... 152
2.3.2. Continuous Case ... 156
2.3.3. Sampling a Linear Function ... 157
2.4. 2.4 Transforming between Distributions ... 159
2.4.1. Transformation in Multiple Dimensions ... 160
2.4.2. Sampling with Multidimensional Transformations ... 161
2.4.2.1. Sampling the Bilinear Function ... 162
2.5. Further Reading ... 165

3. Geometry and Transformations ... 169
3.1. Coordinate Systems ... 170
3.1.1. Coordinate System Handedness ... 171
3.2. n-Tuple Base Classes ... 173
3.3. Vectors ... 178
3.3.1. 3.3.1 Normalization and Vector Length ... 180
3.3.2. 3.3.2 Dot and Cross Product ... 182
3.3.3. Coordinate System from a Vector ... 186
3.4. Points ... 188
3.5. Normals ... 191
3.6. Rays ... 193
3.6.1. Ray Differentials ... 195
3.7. Bounding Boxes ... 198
3.8. Spherical Geometry ... 210
3.8.1. Solid Angles ... 210
3.8.2. Spherical Polygons ... 211
3.8.3. Spherical Parameterizations ... 213
3.8.3.1. Spherical Coordinates ... 213
3.8.3.2. Octahedral Encoding ... 216
3.8.3.3. Equal-Area Mapping ... 220
3.8.4. Bounding Directions ... 224
3.9. Transformations ... 231
3.9.1. Homogeneous Coordinates ... 232

3.9.2. Transform Class Definition ... 233
3.9.3. Basic Operations ... 234
3.9.4. Translations ... 237
3.9.5. Scaling ... 238
3.9.6. x, y, and z Axis Rotations ... 239
3.9.7. Rotation around an Arbitrary Axis ... 241
3.9.8. Rotating One Vector to Another ... 243
3.9.9. The Look-at Transformation ... 245
3.10. Applying Transformations ... 248
3.10.1. Points ... 248
3.10.2. Vectors ... 249
3.10.3. Normals ... 249
3.10.4. Rays ... 250
3.10.5. Bounding Boxes ... 250
3.10.6. Composition of Transformations ... 251
3.10.7. Transformations and Coordinate System Handedness ... 252
3.10.8. Vector Frames ... 252
3.11. Interactions ... 256
3.11.1. Surface Interaction ... 260
3.11.2. Medium Interaction ... 267
3.12. Further Reading ... 268
3.13. Exercises ... 271

4. Radiometry, Spectra, and Color ... 274
4.1. Radiometry ... 276
4.1.1. Basic Quantities ... 278
4.1.1.1. Energy ... 278
4.1.1.2. Flux ... 279
4.1.1.3. Irradiance and Radiant Exitance ... 280
4.1.1.4. Intensity ... 282
4.1.1.5. Radiance ... 282
4.1.2. Incident and Exitant Radiance Functions ... 284
4.1.3. Radiometric Spectral Distributions ... 285
4.1.4. Luminance and Photometry ... 286
4.2. Working with Radiometric Integrals ... 289
4.2.1. Integrals over Projected Solid Angle ... 289
4.2.2. Integrals over Spherical Coordinates ... 290
4.2.3. Integrals over Area ... 291
4.3. Surface Reflection ... 294
4.3.1. The BRDF and the BTDF ... 294
4.3.2. The BSSRDF ... 297
4.4. Light Emission ... 299
4.4.1. Blackbody Emitters ... 301
4.4.2. Standard Illuminants ... 304
4.5. Representing Spectral Distributions ... 307
4.5.1. Spectrum Interface ... 307
4.5.2. General Spectral Distributions ... 309
4.5.3. Embedded Spectral Data ... 314
4.5.4. Sampled Spectral Distributions ... 315

4.5.4.1. SampledSpectrum ... 317
4.5.5. SampledWavelengths ... 321
4.5.6. Discussion ... 325
4.6. Color ... 327
4.6.1. XYZ Color ... 328
4.6.1.1. Chromaticity and xyY Color ... 334
4.6.2. RGB Color ... 336
4.6.3. RGB Color Spaces ... 338
4.6.3.1. Standard Color Spaces ... 341
4.6.4. Why Spectral Rendering? ... 343
4.6.5. Choosing the Number of Wavelength Samples ... 345
4.6.6. From RGB to Spectra ... 349
4.6.6.1. Unbounded RGB ... 361
4.6.6.2. RGB Illuminants ... 363
4.7. Further Reading ... 366
4.8. Exercises ... 371

5. Cameras and Film ... 372
5.1. Camera Interface ... 374
5.1.1. Camera Coordinate Spaces ... 377
5.1.2. The CameraBase Class ... 384
5.2. Projective Camera Models ... 389
5.2.1. Orthographic Camera ... 392
5.2.2. Perspective Camera ... 397
5.2.3. The Thin Lens Model and Depth of Field ... 403
5.3. Spherical Camera ... 413
5.4. Film and Imaging ... 419
5.4.1. The Camera Measurement Equation ... 419
5.4.2. Modeling Sensor Response ... 421
5.4.2.1. Chromatic Adaptation and White Balance ... 431
5.4.2.2. Sampling Sensor Response ... 433
5.4.3. Filtering Image Samples ... 436
5.4.4. The Film Interface ... 440
5.4.5. Common Film Functionality ... 444
5.4.6. RGBFilm ... 447
5.4.7. GBufferFilm ... 453
5.5. Further Reading ... 456
5.6. Exercises ... 461

6. Shapes ... 464
6.1. Basic Shape Interface ... 465
6.1.1. Bounding ... 465
6.1.2. Ray-Bounds Intersections ... 466
6.1.3. Intersection Tests ... 472
6.1.4. Intersection Coordinate Spaces ... 473
6.1.5. Sidedness ... 474
6.1.6. Area ... 474
6.1.7. Sampling ... 475
6.2. spheres ... 479
6.2.1. Bounding ... 483

6.2.2. Intersection Tests ... 483
6.2.3. Surface Area ... 495
6.2.4. Sampling ... 496
6.3. Cylinders ... 507
6.3.1. Area and Bounding ... 508
6.3.2. Intersection Tests ... 509
6.3.3. Sampling ... 513
6.4. 6.4 Disks ... 516
6.4.1. Area and Bounding ... 517
6.4.2. Intersection Tests ... 518
6.4.3. Sampling ... 521
6.5. Triangle Meshes ... 523
6.5.1. Mesh Representation and Storage ... 524
6.5.2. Triangle Class ... 533
6.5.3. Ray-Triangle Intersection ... 541
6.6. Billinear Patches ... 569
6.6.1. Intersection Tests ... 574
6.7. Curves ... 593
6.7.1. Bounding Curves ... 593
6.7.2. Intersection Tests ... 595
6.8. Managing Rounding Error ... 605
6.8.1. Floating-Point Arithmetic ... 606
6.8.1.1. Arithmetic Operations ... 611
6.8.1.2. Error Propagation ... 618
6.8.2. Conservative Ray–Bounds Intersections ... 624
6.8.3. Accurate Quadratic Discriminants ... 627
6.8.4. Robust Triangle Intersections ... 627
6.8.5. Bounding Intersection Point Error ... 629
6.8.5.1. Effect of Transformations ... 635
6.8.6. Robust Spawned Ray Origins ... 639
6.8.7. Avoiding Intersections behind Ray Origins ... 643
6.8.7.1. Triangles ... 644
6.8.7.2. Bilinear Patches ... 647
6.8.8. Discussion ... 647
6.9. Further Reading ... 650
6.10. Exercises ... 658

7. Primitives and Intersection Acceleration ... 667
7.1. Primitive Interface and Geometric Primitives ... 668
7.1.1. Geometric Primitives ... 669
7.1.2. Object Instancing and Primitives in Motion ... 677
7.2. Aggregates ... 679
7.3. Bounding_Volume_Hierarchies ... 681
7.3.1. BVH Construction ... 683
7.3.2. The Surface Area Heuristic ... 694
7.3.3. Linear Bounding Volume Hierarchies ... 701
7.3.4. Compact BVH for Traversal ... 714
7.3.5. Bounding and Intersection Tests ... 717
7.4. Further_Reading ... 724

7.4.1. Grids ... 724
7.4.2. Bounding Volume Hierarchies ... 725
7.4.3. kd-trees ... 730
7.4.4. The Surface Area Heuristic ... 731
7.4.5. Other Topics in Acceleration Structures ... 733
7.5. Exercises ... 736

8. Sampling and reconstruction ... 741
8.1. Sampling Theory ... 742
8.1.1. The Frequency Domain and the Fourier Transform ... 744
8.1.2. Ideal Sampling and Reconstruction ... 747
8.1.3. Aliasing ... 751
8.1.4. Understanding Pixels ... 753
8.1.5. Sampling and Aliasing in Rendering ... 755
8.1.5.1. Sources of Aliasing ... 756
8.1.5.2. Adaptive Sampling ... 757
8.1.6. Prefiltering ... 758
8.1.7. Spectral Analysis of Sampling Patterns ... 759
8.2. Sampling and Integration ... 764
8.2.1. Fourier Analysis of Variance ⚠ ... 764
8.2.2. Low Discrepancy and Quasi Monte Carlo ... 770
8.3. Sampling Interface ... 776
8.4. Independent Sampler ... 781
8.5. Stratified Sampler ... 784
8.6. Halton Sampler ⚠ ... 793
8.6.1. Hammersley and Halton Points ... 793
8.6.2. Randomization via Scrambling ... 797
8.6.3. Halton Sampler Implementation ... 804
8.7. Sobol' Samplers ⚠ ... 823
8.7.1. Stratification over Elementary Intervals ... 826
8.7.2. Randomization and Scrambling ... 827
8.7.3. Sobol' Sample Generation ... 829
8.7.4. Global Sobol' Sampler ... 830
8.8. Image Reconstruction ... 854
8.8.1. Filter Interface ... 855
8.8.2. FilterSampler ... 858
8.8.3. Box Filter ... 861
8.8.4. Triangle Filter ... 864
8.8.5. Gaussian Filter ... 865
8.8.6. Mitchell Filter ... 868
8.8.7. Windowed Sinc Filter ... 871
8.9. Further Reading ... 875
8.9.1. Sampling Theory and Aliasing ... 875
8.9.2. Analysis of Monte Carlo Integration ... 876
8.9.3. Sample Generation Algorithms ... 877
8.9.4. Low-Discrepancy Sampling and QMC ... 879
8.9.5. Filtering and Reconstruction ... 882
8.9.6. Adaptive Sampling and Reconstruction ... 883
8.10. Exercises ... 885

9. Reflection Models ... 887
9.1. BSDF Representation ... 890
9.1.1. Geometric Setting and Conventions ... 890
9.1.2. BxDF Interface ... 892
9.1.3. Hemispherical Reflectance ... 897
9.1.4. Delta Distributions in BSDFs ... 898
9.1.5. BSDFs ... 900
9.2. Diffuse Reflection ... 905
9.3. Specular Reflection and Transmission ... 908
9.3.1. Physical Principles ... 908
9.3.2. The Index of Refraction ... 910
9.3.3. The Law of Specular Reflection ... 912
9.3.4. Snell's Law ... 913
9.3.5. The Fresnel Equations ... 919
9.3.6. The Fresnel Equations for Conductors ... 922
9.4. Conductor BRDF ... 927
9.5. Dielectric BSDF ... 931
9.5.1. Thin Dielectric BSDF ... 936
9.5.2. Non-Symmetric Scattering and Refraction ⚠ ... 940
9.6. Roughness Using Microfacet Theory ... 945
9.6.1. The Microfacet Distribution ... 947
9.6.2. The Masking Function ... 950
9.6.3. The Masking-Shadowing Function ... 954
9.6.4. Sampling the Distribution of Visible Normals ... 955
9.6.5. The Torrance–Sparrow Model ... 957
9.6.6. The Half-Direction Transform ... 958
9.7. Rough Dielectric BSDF ... 963
9.8. Measured BSDF ... 966
9.8.1. MeasuredBxDF Definition ... 966
9.8.2. Finding the Initial Microfacet Model ... 974
9.8.3. 9.8.1 Basic Data Structures ... 975
9.9. Scattering from Hair ... 980
9.9.1. 9.9.1 Geometry ... 981
9.9.2. 9.9.4 Absorption in Fibers ... 994
9.9.3. 9.9.5 Azimuthal Scattering ... 999
9.9.3.1. A Note on Reciprocity ... 1005
9.9.4. 9.9.7 Sampling ... 1006
9.10. Further_Reading ... 1013
9.10.1. Layered Materials ... 1016
9.10.2. BSDF (Re-)Parameterization and Acquisition ... 1017
9.10.3. Hair, Fur, and Fibers ... 1019
9.10.4. Glints and Microstructure ... 1024
9.10.5. Wave Optics ... 1026
9.11. Exercises ... 1030

10. Textures and Materials ... 1034
10.1. Texture Sampling and Antialiasing ... 1036
10.1.1. Finding the Texture Sampling Rate ... 1036
10.1.2. asdfa ... 1049

10.1.3. ⚠ Ray Differentials for Specular Reflection and Transmission 1050
10.1.4. Filtering Texture Functions 1057
10.2. Texture Coordinate Generation 1060
10.2.1. (u, v) Mapping 1062
10.2.2. Spherical Mapping 1063
10.2.3. Cylindrical Mapping 1066
10.2.4. Planar Mapping 1067
10.2.5. 3D Mapping 1068
10.3. Texture Interface and Basic Textures 1071
10.3.1. Constant Texture 1072
10.3.2. Scale Texture 1073
10.3.3. Mix Textures 1076
10.4. 10.4 Image Texture 1079
10.4.1. Texture Memory Management 1081
10.4.2. Image Texture Evaluation 1084
10.4.3. MIP Maps 1086
10.4.4. Image Map Filtering 1090
10.5. Material Interface and Implementations 1100
10.5.1. Material Implementations 1108
10.5.1.1. Diffuse Material 1108
10.5.1.2. Dielectric Material 1109
10.5.1.3. Mix Material 1112
10.5.2. Finding the BSDF at a Surface 1115
10.5.3. Normal mapping 1117
10.5.4. Bump Mapping 1121
10.6. Further_Reading 1127
10.7. Exercises 1137

11. Volume Scattering 1138
11.1. Volume Scattering Processes 1139
11.1.1. Absorption 1141
11.1.2. Emission 1143
11.1.3. Out Scattering and Attenuation 1146
11.1.4. In Scattering 1147
11.2. Transmittance 1150
11.2.1. Null scattering 1154
11.3. Phase Functions 1159
11.3.1. The Henyey–Greenstein Phase Function 1161
11.4. Media 1166
11.4.1. Medium Interface 1170
11.4.2. Homogeneous Medium 1174
11.4.3. DDA Majorant Iterator 1177
11.4.4. Uniform Grid Medium 1186
11.4.5. RGB Grid Medium 1191
11.5. Further_Reading 1199
11.6. Exercises 1205

12. Light Sources 1207
12.1. Light Interface 1209
12.1.1. Photometric Light Specification 1215

12.1.2. The LightBase Class ... 1216
12.2. Point Lights ... 1220
12.2.1. Spotlights ... 1222
12.2.2. Texture Projection Lights ... 1226
12.2.3. Goniophotometric Diagram Lights ... 1232
12.3. Distant_Lights ... 1236
12.4. Area lights ... 1240
12.5. Infinite Area Lights ... 1247
12.5.1. Uniform Infinite Lights ... 1248
12.5.2. Image Infinite Lights ... 1251
12.5.3. Portal Image Infinite Lights ... 1258
12.6. Light Sampling ... 1271
12.6.1. Uniform Light Sampling ... 1273
12.6.2. Power Light Sampler ... 1275
12.6.3. BVH Light Sampling ... 1279
12.6.3.1. Bounding Lights ... 1279
12.6.3.2. Bounds for Light Implementations ... 1287
12.6.3.3. Compactly Bounding Lights ... 1291
12.6.3.4. Light Bounding Volume Hierarchies ... 1298
12.7. Further_Reading ... 1312
12.7.1. Illumination from Environment Maps ... 1313
12.7.2. Optimizing Visibility Testing ... 1315
12.7.2.1. Many-Light Sampling ... 1318
12.8. Exercises ... 1320

13. Light Transport I Surface Reflection ... 1323
13.1. 13.1 The Light Transport Equation ... 1324
13.1.1. Basic Derivation ... 1324
13.1.2. Analytic Solutions to the LTE ... 1326
13.1.3. The Surface Form of the LTE ... 1328
13.1.4. Integral over Paths ... 1330
13.1.5. Delta Distributions in the Integrand ... 1332
13.1.6. Partitioning the Integrand ... 1333
13.2. Path Tracing ... 1336
13.2.1. Overview ... 1337
13.2.2. Path Sampling ... 1338
13.2.3. Incremental Path Construction ... 1341
13.2.3.1. Relationship to the RandomWalkIntegrator ... 1342
13.3. A Simple Path Tracer ... 1343
13.4. A Better Path Tracer ... 1354
13.4.1. Path Regularization ... 1371
13.5. Further Reading ... 1375
13.6. Exercises ... 1389

14. Light Transport II: Volume Rendering ... 1392
14.1. The Equation of Transfer ... 1393
14.1.1. Null-Scattering Extension ... 1395
14.1.2. Evaluating the Equation of Transfer ... 1396
14.1.3. Sampling the Majorant Transmittance ... 1398
14.1.4. Generalized Path Space ⚠ ... 1410

14.1.5. Evaluating the Volumetric Path Integral ⚠ ... 1414
14.2. 14.2 Volume Scattering Integrators ... 1417
14.2.1. A Simple Volumetric Integrator ... 1417
14.2.2. Improving the Sampling Techniques ⚠ ... 1428
14.2.2.1. Chromatic Media ... 1428
14.2.2.2. Direct Lighting ... 1431
14.2.3. Improved Volumetric Integrator ⚠ ... 1434
14.2.3.1. Estimating Direct Illumination ... 1448
14.3. Scattering from Layered Materials ... 1461
14.3.1. The One-Dimensional Equation of Transfer ... 1462
14.3.2. Layered BxDF ... 1464
14.3.2.1. BSDF Evaluation ... 1469
14.3.2.2. BSDF Sampling ... 1480
14.3.2.3. PDF Evaluation ... 1481
14.3.3. Coated Diffuse and Coated Conductor Materials ... 1486
14.4. Further Reading ... 1488
14.5. Exercises ... 1497

15. Wavefront Rendering on GPUs ⚠ ... 1499
15.1. Mapping Path Tracing to the GPU ... 1502
15.1.1. Basic GPU Architecture ... 1502
15.1.1.1. Thread Execution ... 1504
15.1.1.2. Memory Hierarchy ... 1507
15.1.2. Structuring Rendering Computation ... 1513
15.1.3. System Overview ... 1516
15.2. Implementation Foundations ... 1520
15.2.1. Execution and Memory Space Specification ... 1520
15.2.2. Launching Kernels on the GPU ... 1521
15.2.3. Structure-of-Arrays Layout ... 1523
15.2.4. Work Queues ... 1530
15.3. 15.3 Path Tracer Implementation ... 1537
15.3.1. 15.3.9 Surface Scattering ... 1537
15.3.2. 15.3.6 Intersection Testing ... 1537
15.5. 光线追踪硬件 ... 1542
15.7. 练习 ... 1544

16. Retrospective and the Future ... 1548
16.1. pbrt over the Years ... 1549
16.2. Design Alternatives ... 1552
16.2.1. Out-of-Core Rendering ... 1552
16.2.2. Preshaded Micropolygon Grids ... 1554
16.2.3. Packet Tracing ... 1556
16.2.4. Interactive and Animation Rendering ... 1559
16.2.5. Specialized Compilation ... 1560
16.3. Emerging Topics ... 1562
16.3.1. Inverse and Differentiable Rendering ... 1562
16.3.2. Machine Learning and Rendering ... 1571

A. Sampling Algorithms ... 1578

Bibliography ... 1579

1. Perface

[Just as] other information should be available to those who want to learn and understand, program source code is the only means for programmers to learn the art from their predecessors. It would be unthinkable for playwrights not to allow other playwrights to read their plays [or to allow them] at theater performances where they would be barred even from taking notes. Likewise, any good author is well read, as every child who learns to write will read hundreds of times more than it writes. Programmers, however, are expected to invent the alphabet and learn to write long novels all on their own. Programming cannot grow and learn unless the next generation of programmers has access to the knowledge and information gathered by other programmers before them. —Erik Naggum

[正如]那些想要学习和理解的人应该可以获得其它信息一样，源码是程序员向前辈学习艺术的唯一手段。对于剧作家来说，不允许其他剧作家朗读他们的剧本（或允许他们）在剧院演出，甚至不允许他们做笔记，这是无法想象的。同样，任何优秀的作家都善于阅读，因为每个学习写作的孩子所读的内容都会比所写的内容多数百倍。然而，程序员应该发明字母表并学会自己写长篇小说。除非下一代程序员能够获得之前其他程序员收集的知识和信息，否则编程就无法成长和学习。 ——埃里克·纳古姆

Rendering is a fundamental component of computer graphics. At the highest level of abstraction, rendering is the process of converting a description of a three-dimensional scene into an image. Algorithms for animation, geometric modeling, texturing, and other areas of computer graphics all must pass their results through some sort of rendering process so that they can be made visible in an image. Rendering has become ubiquitous; from movies to games and beyond, it has opened new frontiers for creative expression, entertainment, and visualization.

渲染是计算机图形学的基本组成部分。在最高抽象层次上，渲染是将三维场景的描述转换为图像的过程。动画、几何建模、纹理和计算机图形学其他领域的算法都必须通过某种渲染过程传递其结果，以便它们可以在图像中可见。渲染已经变得无处不在；从电影到游戏等等，它为创意表达、娱乐和可视化开辟了新的领域。

In the early years of the field, research in rendering focused on solving fundamental problems such as determining which objects are visible from a given viewpoint. As effective solutions to these problems have been found and as richer and more realistic scene descriptions have become available thanks to continued progress in other areas of graphics, modern rendering has grown to include ideas from a broad range of disciplines, including physics and astrophysics, astronomy, biology, psychology and the study of perception, and pure and applied mathematics. The interdisciplinary nature of rendering is one of the reasons that it is such a fascinating area of study.

在该领域的早期，渲染研究的重点是解决基本问题，例如确定从给定视点哪些对象是可见的。随着这些问题的有效解决方案被发现，并且由于图形学其他领域的不断进步，更丰富、更真实的场景描述已经成为可能，现代渲染已经发展到包含来自广泛学科的想法，包括物理学和天体物理学、天文学、生物学、心理学和感知研究，以及纯数学和应用数学。渲染的跨学科性质是它成为如此令人着迷的研究领域的原因之一。

This book presents a selection of modern rendering algorithms through the documented source code for a complete rendering system. Nearly all of the images in this book, including the one on the front cover, were rendered by this software. All of the algorithms that came together to generate these images are described in these pages. The system, `pbrt`, is written using a programming

methodology called *literate programming* that mixes prose describing the system with the source code that implements it. We believe that the literate programming approach is a valuable way to introduce ideas in computer graphics and computer science in general. Often, some of the subtleties of an algorithm can be unclear or hidden until it is implemented, so seeing an actual implementation is a good way to acquire a solid understanding of that algorithm's details. Indeed, we believe that deep understanding of a number of carefully selected algorithms in this manner provides a better foundation for further study of computer graphics than does superficial understanding of many.

本书通过完整渲染系统的记录源代码介绍了一系列现代渲染算法。本书中几乎所有的图像，包括封面上的图像，都是由该软件渲染的。这些页面描述了生成这些图像的所有算法。`pbrt`系统是使用一种称为*文学编程*的编程方法编写的，该方法将描述系统的散文与实现该系统的源代码混合在一起。我们相信，文学编程方法是引入计算机图形学和计算机科学思想的一种有价值的方式。通常，算法的一些微妙之处在实现之前可能是不清楚或隐藏的，因此查看实际实现是深入了解该算法细节的好方法。事实上，我们相信，以这种方式深入理解许多精心挑选的算法，比对肤浅理解算法，为进一步研究计算机图形学提供了更好的基础。

In addition to clarifying how an algorithm is implemented in practice, presenting these algorithms in the context of a complete and nontrivial software system also allows us to address issues in the design and implementation of medium-sized rendering systems. The design of a rendering system's basic abstractions and interfaces has substantial implications for both the elegance of the implementation and the ability to extend it later, yet the trade-offs in this design space are rarely discussed.

除了阐明算法在实践中如何实现之外，在完整且重要的软件系统背景下呈现这些算法还使我们能够解决中型渲染系统的设计和实现中的问题。渲染系统的基本抽象和接口的设计对于实现的优雅性和可扩展性都具有重大影响，但很少有关此设计的权衡的讨论。

`pbrt` and the contents of this book focus exclusively on photorealistic rendering, which can be defined variously as the task of generating images that are indistinguishable from those that a camera would capture in a photograph or as the task of generating images that evoke the same response from a human observer as looking at the actual scene. There are many reasons to focus on photorealism. Photorealistic images are crucial for special effects in movies because computer-generated imagery must often be mixed seamlessly with footage of the real world. In applications like computer games where all of the imagery is synthetic, photorealism is an effective tool for making the observer forget that he or she is looking at an environment that does not actually exist. Finally, photorealism gives a reasonably well-defined metric for evaluating the quality of the rendering system's output.

`pbrt`的内容专注于照片级真实感的渲染，也可以说是为生成与相机中捕获的图像无法区分的图像的任务，或者生成能够引起人类观察者在看着实际场景做出相同反应的图像的任务。关注照片级写实的原因有很多。逼真的图像对于电影特效至关重要，因为计算机生成的图像通常必须与现实世界的镜头无缝混合。在计算机游戏等所有图像都是合成的应用中，照片级写实主义是一种有效的效果，可以让观察者忘记他或她正在观看一个实际上并不存在的环境。最后，照片级写实系统提供了一个相当明确的指标来评估渲染系统输出的质量。

1.1. Audience

There are three main audiences that this book is intended for. The first is students in graduate or upper-level undergraduate computer graphics classes. This book assumes existing knowledge of computer graphics at the level of an introductory college-level course, although certain key concepts such as basic vector geometry and transformations will be reviewed here. For students who do not have experience with programs that have tens of thousands of lines of source code, the literate programming style gives a gentle introduction to this complexity. We pay special attention to explaining the reasoning behind some of the key interfaces and abstractions in the system in order to give these readers a sense of why the system is structured in the way that it is.

本书主要面向三个读者。第一种是研究生或高年级本科生计算机图形学课程的学生。本书假设计算机图形学的现有知识处于大学入门课程的水平，尽管这里将回顾某些关键概念，例如基本向量几何和变换。对于没有使用过数万行源代码的程序经验的学生来说，文学编程风格可以温和地介绍这种复杂性。我们特别注重解释系统中一些关键接口和抽象背后的推理，以便让读者了解系统为何如此构造。

The second audience is advanced graduate students and researchers in computer graphics. For those doing research in rendering, the book provides a broad introduction to the area, and the pbrt source code provides a foundation that can be useful to build upon (or at least to use bits of source code from). For those working in other areas of computer graphics, we believe that having a thorough understanding of rendering can be helpful context to carry along.

第二类受众是计算机图形学领域的高级研究生和研究人员。对于那些从事渲染研究的人来说，本书提供了该领域的广泛介绍，以及 `pbrt` 源代码提供了一个可用于构建（或至少使用其中的源代码位）的基础。对于那些在计算机图形学其他领域工作的人来说，我们相信对渲染的透彻理解可以为他们提供帮助。

Our final audience is software developers in industry. Although many of the basic ideas in this book will be familiar to this audience, seeing explanations of the algorithms presented in the literate style may lead to new perspectives. `pbrt` also includes carefully crafted and debugged implementations of many algorithms that can be challenging to implement correctly; these should be of particular interest to experienced practitioners in rendering. We hope that delving into one particular organization of a complete and nontrivial rendering system will also be thought provoking to this audience.

我们的最终受众是工业界的软件开发人员。尽管本书中的许多基本思想对于读者来说是熟悉的，但看到以文学风格呈现的算法解释可能会带来新的视角。 `pbrt` 还包括许多算法的精心设计和调试实现，这些算法的正确实现可能具有挑战性；这些应该是经验丰富的渲染从业者特别感兴趣的。我们希望也能引起某个研究完整且重要的渲染系统的组织的深入思考。

1.2. Overview and Goals

`pbrt` is based on the *ray-tracing* algorithm. Ray tracing is an elegant technique that has its origins in lens making; Carl Friedrich Gauß traced rays through lenses by hand in the 19th century. Ray-tracing algorithms on computers follow the path of infinitesimal rays of light through the scene until they intersect a surface. This approach gives a simple method for finding the first visible object as seen from any particular position and direction and is the basis for many rendering algorithms.

`pbrt` 基于光线追踪算法。光线追踪是一项优雅的技术，起源于镜头制造。 19 世纪，卡尔·弗里德里希·高斯 (Carl Friedrich Gauß) 手动追踪穿过透镜的光线。计算机上的光线追踪算法跟踪穿过场景的无穷小光线的路径，直到它们与表面相交。这就提供了一种简单的方法来查找从任何特定位置和方向看到的第一个可见对象，并且是许多渲染算法的基础。

`pbrt` was designed and implemented with three main goals in mind: it should be *complete*, it should be *illustrative*, and it should be *physically based*.

`pbrt` 设计和实现时考虑了三个主要目标：它应该是完整的，它应该是说明性的，并且它应该是基于物理的。

Completeness implies that the system should not lack key features found in high-quality commercial rendering systems. In particular, it means that important practical issues, such as antialiasing, robustness, numerical precision, and the ability to efficiently render complex scenes should all be addressed thoroughly. It is important to consider these issues from the start of the system's design, since these features can have subtle implications for all components of the system and can be quite difficult to retrofit into the system at a later stage of implementation.

完整性意味着系统不应缺少高质量商业渲染系统中的关键功能。特别是，这意味着重要的实际问题，例如抗锯齿、鲁棒性、数值精度以及有效渲染复杂场景的能力都应该得到完整解决。从系统设计之初就考虑这些问题非常重要，因为这些功能可能会对系统的所有组件产生微妙的影响，并且很难在后期实施阶段对系统进行改造。

Our second goal means that we tried to choose algorithms, data structures, and rendering techniques with care and with an eye toward readability and clarity. Since their implementations will be examined by more readers than is the case for other rendering systems, we tried to select the most elegant algorithms that we were aware of and implement them as well as possible. This goal also required that the system be small enough for a single person to understand completely. We have implemented `pbrt` using an extensible architecture, with the core of the system implemented in terms of a set of carefully designed interface classes, and as much of the specific functionality as possible in implementations of these interfaces. The result is that one does not need to understand all of the specific implementations in order to understand the basic structure of the system. This makes it easier to delve deeply into parts of interest and skip others, without losing sight of how the overall system fits together.

我们的第二个目标意味着我们尝试谨慎选择算法、数据结构和渲染技术，并着眼于可读性和清晰度。由于与其他渲染系统相比，它们的实现将受到更多读者的检查，因此我们尝试选择我们所知道的最优雅的算法并尽可能地实现它们。这个目标还要求系统足够小，足以让一个人完全理解。我们已经实现了在`pbrt`中能够可扩展的体系结构，系统的核心通过一组精心设计的接口类来实现，并在这些接口的实现中尽可能多地实现特定功能。结果是，人们不

需要理解所有的具体实现就可以理解系统的基本结构。这使得更容易深入研究感兴趣的部分并跳过其他部分，而不会忽视整个系统如何组合在一起。

There is a tension between the two goals of being complete and being illustrative. Implementing and describing every possible useful technique would not only make this book unacceptably long, but would also make the system prohibitively complex for most readers. In cases where `pbrt` lacks a particularly useful feature, we have attempted to design the architecture so that the feature could be added without altering the overall system design.

完整和说明性这两个目标之间存在着紧张关系。实现和描述每一种可能有用的技术不仅会使本书变得长得令人难以接受，而且还会使系统对大多数读者来说过于复杂。在这种情况下 `pbrt` 由于一些缺乏特别有用的功能，我们尝试设计架构，以便在不改变整体系统设计的情况下添加该功能（指说明性）。

The basic foundations for physically based rendering are the laws of physics and their mathematical expression. `pbrt` was designed to use the correct physical units and concepts for the quantities it computes and the algorithms it implements. `pbrt` strives to compute images that are *physically correct*; they accurately reflect the lighting as it would be in a real-world version of the scene.[1] One advantage of the decision to use a physical basis is that it gives a concrete standard of program correctness: for simple scenes, where the expected result can be computed in closed form, if `pbrt` does not compute the same result, we know there must be a bug in the implementation. Similarly, if different physically based lighting algorithms in `pbrt` give different results for the same scene, or if `pbrt` does not give the same results as another physically based renderer, there is certainly an error in one of them. Finally, we believe that this physically based approach to rendering is valuable because it is rigorous. When it is not clear how a particular computation should be performed, physics gives an answer that guarantees a consistent result.

基于物理的渲染的基本基础是物理定律及其数学表达式。 `pbrt` 旨在为其计算的数值和实现的算法使用正确的物理单位和概念。`pbrt` 努力计算*物理上正确*的图像；它们准确地反映了场景的真实版本中的照明。[2] 使用物理基础的一个优点是它给出了程序正确性的具体标准：对于简单场景，可以以封闭形式计算预期结果， 如果 `pbrt` 没有计算出相同的结果，那实现中一定存在错误。类似地，如果不同的基于物理的照明算法 `pbrt` 对于同一场景给出不同的结果，或者如果 `pbrt` 没有给出与另一个基于物理的渲染器相同的结果，其中肯定存在错误。最后，我们相信这种基于物理的渲染方法很有价值，因为它很严格。当不清楚如何执行特定计算时，物理学会给出保证结果一致的答案。

Efficiency was given lower priority than these three goals. Since rendering systems often run for many minutes or hours in the course of generating an image, efficiency is clearly important. However, we have mostly confined ourselves to algorithmic efficiency rather than low-level code optimization. In some cases, obvious micro-optimizations take a backseat to clear, well-organized code, although we did make some effort to optimize the parts of the system where most of the computation occurs.

[1] Of course, any computer simulation of physics requires carefully choosing approximations that trade off requirements for fidelity with computational efficiency. See Section 1.2 for further discussion of the choices made in

[2] 当然，任何物理的计算机模拟都需要仔细选择近似值，在保真度要求和计算效率之间进行权衡。有关其中所做选择的进一步讨论，请参阅 Section 1.2

效率的优先级低于这三个目标。由于渲染系统在生成图像的过程中通常运行数分钟或数小时，因此效率显然很重要。然而，我们主要局限于算法效率而不是低级代码优化。在某些情况下，尽管我们确实做出了一些努力来优化系统中发生大部分计算的部分，但明显的微观优化让位于清晰、组织良好的代码。

In the course of presenting `pbrt` and discussing its implementation, we hope to convey some hard-learned lessons from years of rendering research and development. There is more to writing a good renderer than stringing together a set of fast algorithms; making the system both flexible and robust is a difficult task. The system's performance must degrade gracefully as more geometry or light sources are added to it or as any other axis of complexity is stressed.

在呈现过程中 `pbrt` 并讨论其实施，我们希望传达一些从多年的渲染研究和开发之中来之不易的经验教训。编写一个好的渲染器不仅仅是将一组快速算法串在一起；使系统既灵活又健壮是一项艰巨的任务。随着更多几何体或光源的添加，或者任何其他复杂度受到压力，系统的性能必须适度下降。

The rewards for developing a system that addresses all these issues are enormous—it is a great pleasure to write a new renderer or add a new feature to an existing renderer and use it to create an image that could not be generated before. Our most fundamental goal in writing this book was to bring this opportunity to a wider audience. Readers are encouraged to use the system to render the example scenes in the `pbrt` software distribution as they progress through the book. Exercises at the end of each chapter suggest modifications to the system that will help clarify its inner workings and more complex projects to extend the system by adding new features.

开发一个解决所有这些问题的系统的回报是巨大的——编写一个新的渲染器或向现有渲染器添加新功能并使用它来创建以前无法生成的图像是一种巨大的乐趣。我们撰写本书的最根本目标是将这个机会带给更广泛的读者。鼓励读者使用该 `pbrt` 系统随着本书的代码来渲染示例场景。每章末尾的练习建议对系统进行修改，这将有助于阐明其内部工作原理，添加新功能来扩展系统，实现更复杂的项目。

The website for this book is located at `pbrt`.org. This site includes links to the `pbrt` source code, scenes that can be downloaded to render with `pbrt`, and a bug tracker, as well as errata. Any errors in this text that are not listed in the errata can be reported to the email address `authors@pbrt.org`. We greatly value your feedback!

本书的网站位于`pbrt`.org 。该网站包含以下链接 `pbrt` 源代码，可以下载渲染的场景 `pbrt` 、错误跟踪器以及勘误表。勘误表中未列出的本文中的任何错误都可以报告至电子邮件地址`authors@pbrt.org` 。我们非常重视您的反馈！

1.3. Changes Between The First and Second Editions

Six years passed between the publication of the first edition of this book in 2004 and the second edition in 2010. In that time, thousands of copies of the book were sold, and the `pbrt` software was downloaded thousands of times from the book's website. The `pbrt` user base gave us a significant amount of feedback and encouragement, and our experience with the system guided many of the decisions we made in making changes between the version of `pbrt` presented in the first edition and the version in the second edition. In addition to a number of bug fixes, we also made several significant design changes and enhancements:

本书从 2004 年第一版出版到 2010 年第二版出版，历时六年。期间，该书已售出数千册，`pbrt` 该软件从该书的网站上被下载了数千次。`pbrt` 用户群给了我们大量的反馈和鼓励，我们对系统的经验指导了我们在版本之间进行更改时做出的许多决定。除了一些错误修复之外，我们还进行了几项重大的设计更改和增强：

- *Removal of the plugin architecture*: The first version of `pbrt` used a runtime plugin architecture to dynamically load code for implementations of objects like shapes, lights, integrators, cameras, and other objects that were used in the scene currently being rendered. This approach allowed users to extend `pbrt` with new object types (e.g., new shape primitives) without recompiling the entire rendering system. This approach initially seemed elegant, but it complicated the task of supporting `pbrt` on multiple platforms and it made debugging more difficult. The only new usage scenario that it truly enabled (binary-only distributions of `pbrt` or binary plugins) was actually contrary to our pedagogical and open-source goals. Therefore, the plugin architecture was dropped in this edition.
- *删除插件架构*：第一个版本 `pbrt` 使用运行时插件的架构来动态加载对象的实现代码，例如形状、灯光、积分器、相机以及当前渲染场景中使用的其他对象。这种方法允许用户扩展 `pbrt` 无需重新编译整个渲染系统即可使用新的对象类型（例如，新的形状基元）。这种方法最初看起来很优雅，但它使`pbrt`支持多个平台变得复杂，这使得调试变得更加困难。它真正启用的唯一新使用场景（仅`pbrt` 二进制发行版 或二进制插件）实际上违背了我们的教学和开源目标。因此，该版本中删除了插件架构。
- *Removal of the image-processing pipeline*: The first version of `pbrt` provided a tone-mapping interface that converted high-dynamic-range (HDR) floating-point output images directly into low-dynamic-range TIFFs for display. This functionality made sense in 2004, as support for HDR images was still sparse. In 2010, however, advances in digital photography had made HDR images commonplace. Although the theory and practice of tone mapping are elegant and worth learning, we decided to focus the new book exclusively on the process of image formation and skip the topic of image display. Interested readers should consult the book written by Reinhard et al. 2010 for a thorough and modern treatment of the HDR image display process.
- *删除图像处理管道*：第一个版本 `pbrt` 提供了一个色调映射接口，可将高动态范围 (HDR) 浮点输出图像直接转换为低动态范围 TIFF 进行显示。此功能在 2004 年有意义，因为对 HDR 图像的支持仍然很少。然而，到了 2010 年，数字摄影技术的进步使得 HDR 图像变得司空见惯。尽管色调映射的理论和实践很优雅且值得学习，但我们决定将新书的重点放在图像形成的过程上，而跳过图像显示的主题。有兴趣的读者可以查阅 Reinhard 等人写的书。 2010 对 HDR 图像显示过程进行彻底和现代的处理。

- *Task parallelism*: Multicore architectures became ubiquitous, and we felt that `pbrt` would not remain relevant without the ability to scale to the number of locally available cores. We also hoped that the parallel programming implementation details documented in this book would help graphics programmers understand some of the subtleties and complexities in writing scalable parallel code.
- *任务并行性*：多核架构变得无处不在，我们认为 `pbrt` 如果无法扩展到本地可用核心的数量，就无法保持相关性。我们还希望本书中记录的并行编程实现细节能够帮助图形程序员理解编写可扩展的并行代码的一些微妙之处和复杂性。
- *Appropriateness for "production" rendering*: The first version of `pbrt` was intended exclusively as a pedagogical tool and a stepping-stone for rendering research. Indeed, we made a number of decisions in preparing the first edition that were contrary to use in a production environment, such as limited support for image-based lighting, no support for motion blur, and a photon mapping implementation that was not robust in the presence of complex lighting. With much improved support for these features as well as support for subsurface scattering and Metropolis light transport, we feel that with the second edition, `pbrt` became much more suitable for rendering very high-quality images of complex environments.
- *"生产级"渲染的适当性*：第一个版本 `pbrt` 专门作为一种教学工具和渲染研究的垫脚石。事实上，我们在准备第一版时做出了许多与生产环境中相反的决定，例如对基于图像的照明的有限支持、不支持运动模糊以及在实际环境中不稳健的光子映射实现。复杂照明的存在。通过对这些功能的大幅改进支持以及对次表面散射和 Metropolis 光传输的支持，我们认为在第二版中，`pbrt` 变得更适合渲染复杂环境的高质量图像。

1.4. Changes Between The Second and Third Editions

With the passage of another six years, it was time to update and extend the book and the `pbrt` system. We continued to learn from readers' and users' experiences to better understand which topics were most useful to cover. Further, rendering research continued apace; many parts of the book were due for an update to reflect current best practices. We made significant improvements on a number of fronts:

又六年过去了，是时候更新和扩展这本书和 `pbrt` 系统。我们继续从读者和用户的经验中学习，以更好地了解哪些主题最有用。此外，渲染研究继续快速发展；本书的许多部分都需要更新，以反映当前的最佳实践。我们在许多方面做出了重大改进：

- *Bidirectional light transport*: The third version of `pbrt` added a bidirectional path tracer, including full support for volumetric light transport and multiple importance sampling to weight paths. An all-new Metropolis light transport integrator used components of the bidirectional path tracer, allowing for a particularly succinct implementation of that algorithm.
- *双向光传输*：第三个版本 `pbrt` 添加了双向路径跟踪器，包括对体积光传输的全面支持和对权重路径的多重重要性采样。全新的 Metropolis 光传输积分器使用了双向路径追踪器的组件，从而可以特别简洁地实现该算法。
- *Subsurface scattering*: The appearance of many objects—notably, skin and translucent objects—is a result of subsurface light transport. Our implementation of subsurface scattering in the second edition reflected the state of the art in the early 2000s; we thoroughly updated both BSSRDF models and our subsurface light transport algorithms to reflect the progress made in ten subsequent years of research.
- *次表面散射*：许多物体（尤其是皮肤和半透明物体）的外观是次表面光传输的结果。我们在第二版中实现的次表面散射反映了 2000 年代初期的技术水平；我们彻底更新了 BSSRDF 模型和次表面光传输算法，以反映随后十年研究取得的进展。
- *Numerically robust intersections*: The effects of floating-point round-off error in geometric ray intersection calculations have been a long-standing challenge in ray tracing: they can cause small errors to be present throughout the image. We focused on this issue and derived conservative (but tight) bounds of this error, which made our implementation more robust to this issue than previous rendering systems.
- *数值稳定的求交运算*：几何射线相交计算中浮点舍入误差的影响一直是光线追踪中的一个长期挑战：它们可能导致整个图像中出现小错误。我们专注于这个问题并得出了这个错误的保守（但严格）界限，这使得我们的实现比以前的渲染系统对此问题更加稳定。
- *Participating media representation*: We significantly improved the way that scattering media are described and represented in the system; this allows for more accurate results with nested scattering media. A new sampling technique enabled unbiased rendering of heterogeneous media in a way that cleanly integrated with all of the other parts of the system.
- *参与的媒体表示*：我们显着改进了系统中描述和表示散射媒体的方式；这允许使用嵌套散射介质获得更准确的结果。新的采样技术能够以与系统的所有其他部分整洁的集成在一起，实现异构媒体的无偏差渲染。

- *Measured materials*: This edition added a new technique to represent and evaluate measured materials using a sparse frequency-space basis. This approach is convenient because it allows for exact importance sampling, which was not possible with the representation used in the previous edition.
- 测量材料：此版本添加了一种新技术，使用稀疏频率空间基础来表示和评估测量材料。这种方法很方便，因为它允许精确的重要性采样，而这对于以前版本中使用的表示是不可能的。
- *Photon mapping*: A significant step forward for photon mapping algorithms has been the development of variants that do not require storing all of the photons in memory. We replaced `pbrt`'s photon mapping algorithm with an implementation based on stochastic progressive photon mapping, which efficiently renders many difficult light transport effects.
- 光子映射：光子映射算法向前迈出的重要一步是开发了不需要将所有光子存储在内存中的变体。我们更换了 `pbrt` 的光子映射算法采用基于随机渐进光子映射的实现，可有效渲染许多困难的光传输效果。
- *Sample generation algorithms*: The distribution of sample values used for numerical integration in rendering algorithms can have a surprisingly large effect on the quality of the final results. We thoroughly updated our treatment of this topic, covering new approaches and efficient implementation techniques in more depth than before.
- 样本生成算法：渲染算法中用于数值积分的样本值的分布会对最终结果的质量产生令人惊讶的巨大影响。我们彻底更新了对此主题的处理，比以前更深入地涵盖了新方法和高效的实施技术。

Many other parts of the system were improved and updated to reflect progress in the field: microfacet reflection models were treated in more depth, with much better sampling techniques; a new "curve" shape was added for modeling hair and other fine geometry; and a new camera model that simulates realistic lens systems was made available. Throughout the book, we made numerous smaller changes to more clearly explain and illustrate the key concepts in physically based rendering systems like `pbrt`.

系统的许多其他部分都得到了改进和更新，以反映该领域的进展：微面反射模型得到了更深入的处理，采用了更好的采样技术；添加了新的"曲线"形状，用于建模头发和其他精细几何形状；并推出了模拟真实镜头系统的新相机模型。在整本书中，我们做了许多较小的更改，以更清楚地解释和说明基于物理的渲染系统中的关键概念，例如 `pbrt` 。

1.5. Changes Between The Third and Fourth Editions

Innovation in rendering algorithms has shown no sign of slowing down, and so in 2019 we began focused work on a fourth edition of the text. Not only does almost every chapter include substantial additions, but we have updated the order of chapters and ideas introduced, bringing Monte Carlo integration and the basic ideas of path tracing to the fore rather than saving them for the end.

渲染算法的创新没有放缓的迹象，因此我们在 2019 年开始专注于第四版文本的工作。不仅几乎每一章都包含了大量的补充，而且我们还更新了章节和引入的思想的顺序，将蒙特卡洛积分和路径追踪的基本思想放在首位，而不是把它们留到最后。

Capabilities of the system that have seen especially significant improvements include:

该系统的功能得到了显著的改进，包括：

- *Volumetric scattering*: We have updated the algorithms that model scattering from participating media to the state of the art, adding support for emissive volumes, efficient sampling of volumes with varying densities, and robust support for chromatic media, where the scattering properties vary by wavelength.
- *体积散射*：我们将参与介质的散射建模算法更新为最新技术，增加了对发射体积的支持、对不同密度体积的高效采样以及对彩色介质的强大支持，其中散射属性随波长而变化。
- *Spectral rendering*: We have excised all use of RGB color for lighting calculations; `pbrt` now performs lighting calculations exclusively in terms of samples of wavelength-dependent spectral distributions. Not only is this approach more physically accurate than using RGB, but it also allows `pbrt` to accurately model effects like dispersion.
- *光谱渲染*：我们取消了所有用于照明计算的 RGB 颜色；`pbrt` 现在仅根据波长相关光谱分布的样本执行照明计算。这种方法不仅在物理上比使用 RGB 更准确，而且还允许 `pbrt` 准确地模拟色散等效应。
- *Reflection models*: Our coverage of the foundations of BSDFs and reflection models has been extensively revised, and we have expanded the range of BSDFs covered to include one that accurately models reflection from hair and another that models scattering from layered materials. The measured BRDF follows a new approach that can represent a wide set of materials' reflection spectra.
- *反射模型*：我们对 BSDFs 和反射模型的基础进行了广泛的修订，并且我们扩展了 BSDFs 的范围，包括精确模拟头发反射的一个模型和另一个模拟分层材料散射的模型。测量的 BRDFs 采用了一种新方法，可以代表多种材料的反射光谱。
- *Light sampling*: Not only have we improved the algorithms for sampling points on individual light sources to better reflect the state of the art, but this edition also includes support for many-light sampling, which makes it possible to efficiently render scenes with thousands or millions of light sources by carefully sampling just a few of them.
- *光采样*：我们不仅改进了单个光源采样点的算法以更好地反映现有技术，而且该版本还包括对多光采样的支持，这使得高效渲染数千或数百万的场景成为可能，通过仔细采样其中的几个光源来分析光源。

- *GPU rendering*: This version of `pbrt` adds support for rendering on GPUs, which can provide 10–100 times higher ray tracing performance than CPUs. We have implemented this capability in a way so that almost all of the code presented in the book runs on both CPUs and GPUs, which has made it possible to localize discussion of GPU-related issues to Chapter 15 .
- *GPU渲染*：这个版本 `pbrt` 增加了对 GPU 渲染的支持，可以提供比 CPU 高 10-100 倍的光线追踪性能。我们以某种方式实现了这一功能，使得本书中提供的几乎所有代码都可以在 CPU 和 GPU 上运行，这使得将 GPU 相关问题的讨论集中在第 15 章 。

The system has seen numerous other improvements and additions, including a new bilinear patch shape, many updates to the sample-generation algorithms that are at the heart of Monte Carlo integration, support for outputting auxiliary information at each pixel about the visible surface geometry and reflection properties, and many more small improvements to the system.

该系统还进行了许多其他改进和添加，包括新的双线性补丁形状、对蒙特卡罗积分核心的样本生成算法的许多更新、支持在每个像素输出有关可见表面几何形状和反射的辅助信息属性，以及对系统的许多小改进。

1.6. Acknowledgments

Pat Hanrahan has contributed to this book in more ways than we could hope to acknowledge; we owe a profound debt to him. He tirelessly argued for clean interfaces and finding the right abstractions to use throughout the system, and his understanding of and approach to rendering deeply influenced its design. His willingness to use `pbrt` and this manuscript in his rendering course at Stanford was enormously helpful, particularly in the early years of its life when it was still in very rough form; his feedback throughout this process has been crucial for bringing the text to its current state. Finally, the group of people that Pat helped assemble at the Stanford Graphics Lab, and the open environment that he fostered, made for an exciting, stimulating, and fertile environment. Matt and Greg both feel extremely privileged to have been there.

帕特·汉拉汉 (Pat Hanrahan) 对本书的贡献超出了我们的预期。我们深受他的恩惠。他孜孜不倦地主张干净的界面并找到在整个系统中使用的正确抽象，他对渲染的理解和方法深刻地影响了其设计。他愿意使用 `pbrt` 这份手稿在他在斯坦福大学的渲染课程中非常有帮助，特别是在它的早期阶段，当时它的形式还很粗糙；他在整个过程中的反馈对于使案文达到目前的状态至关重要。最后，帕特帮助在斯坦福图形实验室召集的一群人，以及他培育的开放环境，创造了一个令人兴奋、刺激和肥沃的环境。马特和格雷格都对能够来到那里感到非常荣幸。

We owe a debt of gratitude to the many students who used early drafts of this book in courses at Stanford and the University of Virginia between 1999 and 2004. These students provided an enormous amount of feedback about the book and `pbrt`. The teaching assistants for these courses deserve special mention: Tim Purcell, Mike Cammarano, Ian Buck, and Ren Ng at Stanford, and Nolan Goodnight at Virginia. A number of students in those classes gave particularly valuable feedback and sent bug reports and bug fixes; we would especially like to thank Evan Parker and Phil Beatty. A draft of the manuscript of this book was used in classes taught by Bill Mark and Don Fussell at the University of Texas, Austin, and Raghu Machiraju at Ohio State University; their feedback was invaluable, and we are grateful for their adventurousness in incorporating this system into their courses, even while it was still being edited and revised.

我们要感谢许多学生，他们在 1999 年至 2004 年间在斯坦福大学和弗吉尼亚大学的课程中使用了本书的早期草稿。这些学生提供了关于本书的大量反馈， `pbrt` 。这些课程的助教值得特别提及：斯坦福大学的 Tim Purcell、Mike Cammarano、Ian Buck 和 Ren Ng，以及弗吉尼亚大学的 Nolan Goodnight。这些课程中的许多学生提供了特别有价值的反馈，并发送了错误报告和错误修复；我们要特别感谢埃文·帕克和菲尔·比蒂。德克萨斯大学奥斯汀分校的 Bill Mark 和 Don Fussell 以及俄亥俄州立大学的 Raghu Machiraju 教授的课程中使用了本书的手稿草稿；他们的反馈非常宝贵，我们感谢他们冒险地将这个系统纳入他们的课程，即使该系统仍在编辑和修订中。

Matt Pharr would like to acknowledge colleagues and co-workers in rendering-related endeavors who have been a great source of education and who have substantially influenced his approach to writing renderers and his understanding of the field. Particular thanks go to Craig Kolb, who provided a cornerstone of Matt's early computer graphics education through the freely available source code to the rayshade ray-tracing system, and Eric Veach, who has also been generous with his time and expertise. Thanks also to Doug Shult and Stan Eisenstat for formative lessons in mathematics and computer science during high school and college, respectively, and most important to Matt's parents, for the education they have provided and continued encouragement along the way.

Finally, thanks to NVIDIA for supporting the preparation of both the first and this latest edition of the book; at NVIDIA, thanks to Nick Triantos and Jayant Kolhe for their support through the final stages of the preparation of the first edition and thanks to Aaron Lefohn, David Luebke, and Bill Dally for their support of work on the fourth edition.

Matt Pharr 衷心感谢从事渲染相关工作的同事，他们为他提供了重要的教育来源，并极大地影响了他编写渲染器的方法以及他对该领域的理解。特别感谢 Craig Kolb，他通过免费提供源代码为 Matt 的早期计算机图形学教育奠定了基石。 rayshade 光线追踪系统，以及 Eric Veach，他也慷慨地贡献了自己的时间和专业知识。还要感谢道格·舒尔特 (Doug Shult) 和斯坦·艾森斯塔特 (Stan Eisenstat) 分别在高中和大学期间开设的数学和计算机科学课程，最重要的是马特的父母，感谢他们一路上提供的教育和持续的鼓励。最后，感谢 NVIDIA 对本书第一版和最新版的准备工作的支持；感谢 NVIDIA 的 Nick Triantos 和 Jayant Kolhe 在第一版准备的最后阶段提供的支持，并感谢 Aaron Lefohn、David Luebke 和 Bill Dally 对第四版工作的支持。

Greg Humphreys is very grateful to all the professors and TAs who tolerated him when he was an undergraduate at Princeton. Many people encouraged his interest in graphics, specifically Michael Cohen, David Dobkin, Adam Finkelstein, Michael Cox, Gordon Stoll, Patrick Min, and Dan Wallach. Doug Clark, Steve Lyon, and Andy Wolfe also supervised various independent research boondoggles without even laughing once. Once, in a group meeting about a year-long robotics project, Steve Lyon became exasperated and yelled, "Stop telling me why it can't be done, and figure out how to do it!"—an impromptu lesson that will never be forgotten. Eric Ristad fired Greg as a summer research assistant after his freshman year (before the summer even began), pawning him off on an unsuspecting Pat Hanrahan and beginning an advising relationship that would span 10 years and both coasts. Finally, Dave Hanson taught Greg that literate programming was a great way to work and that computer programming can be a beautiful and subtle art form.

格雷格·汉弗莱斯非常感谢所有在他就读普林斯顿大学本科时包容他的教授和助教。许多人激发了他对图形的兴趣，特别是迈克尔·科恩、大卫·多布金、亚当·芬克尔斯坦、迈克尔·考克斯、戈登·斯托尔、帕特里克·明和丹·瓦拉赫。道格·克拉克（Doug Clark）、史蒂夫·里昂（Steve Lyon）和安迪·沃尔夫（Andy Wolfe）也监督过各种毫无意义的独立研究，甚至没有笑过一次。有一次，在一次关于为期一年的机器人项目的小组会议上，史蒂夫·里昂（Steve Lyon）变得愤怒并大喊："别再告诉我为什么做不到，而是想办法去做！"——这是一个永远被忘掉的偶然教训。埃里克·里斯塔德（Eric Ristad）在大一结束后（甚至在夏天开始之前）解雇了格雷格（Greg），让他担任暑期研究助理，将他抵押给毫无戒心的帕特·汉拉汉（Pat Hanrahan），并开始了一段长达十年的顾问关系。最后，戴夫·汉森告诉格雷格，文学编程是一种很好的工作方式，计算机编程可以是一种美丽而微妙的艺术形式。

Wenzel Jakob was excited when the first edition of `pbrt` arrived in his mail during his undergraduate studies in 2004. Needless to say, this had a lasting effect on his career—thus Wenzel would like to begin by thanking his co-authors for inviting him to become a part of the third and fourth editions of this book. Wenzel is extremely indebted to Steve Marschner, who was his Ph.D. advisor during a fulfilling five years at Cornell University. Steve brought him into the world of research and remains a continuous source of inspiration. Wenzel is also thankful for the guidance and stimulating research environment created by the other members of the graphics group, including Kavita Bala, Doug James, and Bruce Walter. Wenzel spent a wonderful postdoc with Olga Sorkine Hornung,

who introduced him to geometry processing. Olga's support for Wenzel's involvement in the third edition of this book is deeply appreciated.

2004 年，当第一版 `pbrt` 发布时，温泽尔·雅各布 (Wenzel Jakob) 非常兴奋。 他在读本科期间收到了这本书。不用说，这对他的职业生涯产生了持久的影响，因此 Wenzel 首先要感谢他的合著者邀请他参与第三版和第四版的一部分。Wenzel 非常感谢 Steve Marschner，他是他的博士。在康奈尔大学度过了充实的五年期间的顾问。史蒂夫把他带入了研究领域，并一直是他源源不断的灵感源泉。 Wenzel 还感谢图形小组其他成员（包括 Kavita Bala、Doug James 和 Bruce Walter）所创造的指导和激励性的研究环境。 Wenzel 与 Olga Sorkine Hornung 一起度过了一段精彩的博士后时光，Olga Sorkine Hornung 向他介绍了几何处理。奥尔加对温泽尔参与本书第三版的支持深表感谢。

We would especially like to thank the reviewers who read drafts in their entirety; all had insightful and constructive feedback about the manuscript at various stages of its progress. For providing feedback on both the first and second editions of the book, thanks to Ian Ashdown, Per Christensen, Doug Epps, Dan Goldman, Eric Haines, Erik Reinhard, Pete Shirley, Peter-Pike Sloan, Greg Ward, and a host of anonymous reviewers. For the second edition, thanks to Janne Kontkanen, Bill Mark, Nelson Max, and Eric Tabellion. For the fourth edition, we are grateful to Thomas Müller and Per Christensen, who both offered extensive feedback that has measurably improved the final version.

我们要特别感谢审阅全文的审稿人；在手稿进展的各个阶段，所有人都提出了富有洞察力和建设性的反馈。感谢 Ian Ashdown、Per Christensen、Doug Epps、Dan Goldman、Eric Haines、Erik Reinhard、Pete Shirley、Peter-Pike Sloan、Greg Ward 以及许多匿名人士对本书第一版和第二版提供的反馈审稿人。第二版感谢 Janne Kontkanen、Bill Mark、Nelson Max 和 Eric Tabellion。对于第四版，我们感谢 Thomas Müller 和 Per Christensen，他们提供了广泛的反馈，极大地改进了最终版本。

Many experts have kindly explained subtleties in their work to us and guided us to best practices. For the first and second editions, we are also grateful to Don Mitchell, for his help with understanding some of the details of sampling and reconstruction; Thomas Kollig and Alexander Keller, for explaining the finer points of low-discrepancy sampling; Christer Ericson, who had a number of suggestions for improving our kd-tree implementation; and Christophe Hery and Eugene d'Eon for helping us with the nuances of subsurface scattering.

许多专家向我们亲切地解释了他们工作中的微妙之处，并指导我们采取最佳实践。对于第一版和第二版，我们还要感谢 Don Mitchell，他帮助我们理解了采样和重建的一些细节；Thomas Kollig 和 Alexander Keller，解释了低差异采样的细节； Christer Ericson，他对改进我们的 kd-tree 实现提出了许多建议； Christophe Hery 和 Eugene d'Eon 帮助我们解决了次表面散射的细微差别。

For the third edition, we would especially like to thank Leo Grünschloß for reviewing our sampling chapter; Alexander Keller for suggestions about topics for that chapter; Eric Heitz for extensive help with details of microfacets and reviewing our text on that topic; Thiago Ize for thoroughly reviewing the text on floating-point error; Tom van Bussel for reporting a number of errors in our BSSRDF code; Ralf Habel for reviewing our BSSRDF text; and Toshiya Hachisuka and Anton Kaplanyan for extensive review and comments about our light transport chapters.

对于第三版，我们特别感谢 Leo Grünschloß 审阅了我们的抽样章节；亚历山大·凯勒（Alexander Keller）提供有关该章主题的建议； Eric Heitz 在微方面的细节方面提供了广泛的帮助，并审阅了我们关于该主题的文本； Thiago Ize 彻底审阅了有关浮点错误的文本； Tom van Bussel 报告了我们的 BSSRDF 代码中的一些错误；Ralf Habel 审阅了我们的 BSSRDF 文本；Toshiya Hachisuka 和 Anton Kaplanyan 对我们的光传输章节进行了广泛的审查和评论。

For the fourth edition, thanks to Alejandro Conty Estevez for reviewing our treatment of many-light sampling; Eugene d'Eon, Bailey Miller, and Jan Novák for comments on the volumetric scattering chapters; Eric Haines, Simon Kallweit, Martin Stich, and Carsten Wächter for reviewing the chapter on GPU rendering; Karl Li for feedback on a number of chapters; Tzu-Mao Li for his review of our discussion of inverse and differentiable rendering; Fabrice Rousselle for feedback on machine learning and rendering; and Gurprit Singh for comments on our discussion of Fourier analysis of Monte Carlo integration. We also appreciate extensive comments and suggestions from Jeppe Revall Frisvad on `pbrt`'s treatment of reflection models in previous editions.

对于第四版，感谢 Alejandro Conty Estevez 审查我们对多光源采样的处理；Eugene d'Eon、Bailey Miller 和 Jan Novák 对体积散射章节的评论； Eric Haines、Simon Kallweit、Martin Stich 和 Carsten Wächter 审阅了有关 GPU 渲染的章节； Karl Li 对多个章节的反馈； Tzu-Mao Li 对我们关于逆渲染和可微渲染的讨论进行了回顾；Fabrice Rousselle 提供有关机器学习和渲染的反馈； Gurprit Singh 对我们讨论蒙特卡罗积分的傅里叶分析的评论。我们也感谢 Jeppe Revall Frisvad 的广泛评论和建议 `pbrt` 先前版本中对反射模型的处理。

For improvements to `pbrt`'s implementation in this edition, thanks to Pierre Moreau for his efforts in debugging `pbrt`'s GPU support on Windows and to Jim Price, who not only found and fixed numerous bugs in the early release of `pbrt`'s source code, but who also contributed a better representation of chromatic volumetric media than our original implementation. We are also very appreciative of Anders Langlands and Luca Fascione of Weta Digital for providing an implementation of their *PhysLight* system, which has been incorporated into `pbrt`'s `PixelSensor` class and light source implementations.

为了改进 `pbrt` 本版的实现，感谢 Pierre Moreau 在调试 `pbrt` 支持 Windows 上的 GPU 方面的努力 以及 Jim Price，他不仅在早期版本中发现并修复了许多错误 的 `pbrt` 源代码，但他也贡献了比我们最初的实现更好的彩色体积媒体。我们也非常感谢 Weta Digital 的 Anders Langlands 和 Luca Fascione 提供了 *PhysLight* 系统的实现，该系统已并入 `pbrt` 的 `PixelSensor` 类和光源实现。

Many people have reported errors in the text of previous editions or bugs in `pbrt`. We'd especially like to thank Solomon Boulos, Stephen Chenney, Per Christensen, John Danks, Mike Day, Kevin Egan, Volodymyr Kachurovskyi, Kostya Smolenskiy, Ke Xu, and Arek Zimny, who have been especially prolific.

许多人报告了以前版本的文本错误或错误 `pbrt` 。我们要特别感谢 Solomon Boulos、Stephen Chenney、Per Christensen、John Danks、Mike Day、Kevin Egan、Volodymyr Kachurovskyi、Kostya Smolenskiy、Ke Xu 和 Arek Zimny，他们的成果尤其丰富。

For their suggestions and bug reports, we would also like to thank Rachit Agrawal, Frederick Akalin, Thomas de Bodt, Mark Bolstad, Brian Budge, Jonathon Cai, Bryan Catanzaro, Tzu-Chieh Chang,